

AI 코딩테스트 기능 정리

사용자가 완료한 레슨을 기준으로 AI가 문제를 생성하고, 서버가 생성 결과와 사용자의 Java 코드를 같은 실행 환경에서 검증·채점하는 기능이다.

프로젝트	에이콘 E-학습터 Knowva
담당	이현겸
문서 범위	AI 문제 생성 · 실행 테스트 · 채점 · 다음 레벨 해금
기준일	2026. 07. 23.

01 · GENERATION PIPELINE

학습 범위 안에서만 문제를 생성

ExamLearningScopeService가 사용자·과목·난이도별 완료 레슨을 읽어 학습 범위를 만든다. AI 응답은 그대로 저장하지 않고, JSON 구조·허용 API·테스트케이스·정답 코드 실행 결과를 서버에서 다시 검증한다.



```
String solutionCode = requiredText(problem, "solutionCode");
validateSolutionCode(solutionCode, learningScopeText);

String testCaseSpec = objectMapper.writeValueAsString(
    validTestCases(problem));
validateSolutionExecution(solutionCode, testCaseSpec);

parsed.add(new GeneratedProblem(
    ExamPromptNormalizer.normalize(requiredText(problem, "prompt")),
    requiredText(problem, "title"),
    testCaseSpec,
    solutionCode));
```

생성 형식	문제 목록 JSON을 파싱한다. 미완성 TODO, Solution 클래스 누락, 잘못된 테스트케이스는 생성 실패로 처리한다.
학습 범위	Scanner 등 현재까지 학습하지 않은 API가 정답 코드에 포함되면 생성 결과를 거절한다.
재시도	문제 생성은 최대 2회 시도하며, 최종 실패 시 시험 세션 상태를 FAILED로 남긴다.

02 · EXECUTION SAFETY

코드 실행은 시간·메모리·API 범위를 제한

실행 테스트와 최종 채점은 모두 `TestCaseExecutionService`를 사용한다. 생성된 정답 코드도 같은 실행기를 통과해야만 문제로 저장되므로, 출제 결과와 채점 기준이 어긋나는 문제를 줄인다.

```
private static final Duration COMPILE_TIMEOUT = Duration.ofSeconds(10);
private static final Duration RUN_TIMEOUT = Duration.ofSeconds(3);
private static final int MAX_CASE_COUNT = 20;
private static final int MAX_OUTPUT_BYTES = 16_384;

ProcessResult compile = runProcess(workDir, COMPILE_TIMEOUT,
    "javac", "--limit-modules", "java.base", "Solution.java");

ProcessResult run = runProcessWithInput(workDir, input, RUN_TIMEOUT,
    "java", "--limit-modules", "java.base",
    "-Xms16m", "-Xmx64m", "Solution");
```

실행 환경의 경계

임시 디렉터리에 `Solution.java`만 작성하고, `java.base` 모듈·제한된 환경 변수·출력 크기·시간 제한을 적용한다. 실행 후 임시 디렉터리는 정리한다.

학습자 코드와 정답 코드의 분리

```
public static String starterCode(AiExamProblem ignoredProblem) {
    return DEFAULT_STARTER_CODE;
}
```

AI raw response에 정답 로직이 들어 있어도 학습자 에디터와 분석 AI에는 서버 공통 TODO 골격만 전달한다. 서버만 raw response에서 `solutionCode`를 추출해 채점과 시연용 마법 버튼에 사용한다.

03 · SUBMISSION & UNLOCK

동일 기준으로 채점하고 다음 레벨을 해금

제출 시 세 문제의 답안을 각각 실행해 통과한 테스트케이스 수, 정오답, 실행 결과, AI 피드백을 기록한다. 3문제 중 2문제 이상 통과하면 다음 난이도를 해금한다.

```
int correctCount = 0;
for (AiExamProblem problem : problems) {
    ExamAnswer answer = answers.get(problem.getAiProblemId());
    TestCaseExecutionResult executionResult =
        testCaseExecutionService.execute(problem, answer);
    answer.setPassedCaseCount(executionResult.passedCount());
    answer.setIsCorrect(executionResult.passed());
    answer.setTestCaseResult(toJson(executionResult));
    examAnswerMapper.updateGradingResult(answer);
    if (executionResult.passed()) correctCount++;
}

session.setResultStatus(correctCount >= PASS_COUNT ? "PASSED" : "FAILED");
if (correctCount >= PASS_COUNT) {
    unlockService.unlockNextLevel(userId, subjectId, levelCode, examId);
}
```

실행 테스트	세션 소유자와 응시 가능한 상태를 확인한 뒤 코드를 실행한다. 실행 이력이 있어야 UI에서 제출을 활성화한다.
최종 제출	실행기 결과를 재사용하지 않고 제출 시점에 다시 실행한다. 화면의 임시 결과만으로 합격이 결정되지 않는다.
검증	AiGeneratedProblemParserTest, TestCaseExecutionServiceTest, ExamCodeRunServiceAuthorizationTest로 생성·실행·권한 경계를 확인한다.

exam/service/AiExamService.java
exam/service/AiGeneratedProblemParser.java
exam/service/TestCaseExecutionService.java
exam/support/ExamStarterCodeResolver.java